Your Project Structure

```
RUN-WAY/
├── backend/
│   ├── index.js          ← Main server entry
│   ├── middleware/
│   │   └── auth.js       ← JWT authentication
│   ├── routes/
│   │   ├── admin.js      ← Admin routes
│   │   └── (uploads.js?) ← File upload routes
│   ├── uploads/          ← Local image storage (⚠ should move to Cloudinary)
│   ├── .env              ← Secrets (gitignored ✓)
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── main.jsx      ← React entry point
│   │   ├── App.jsx       ← Router & layout
│   │   ├── App.css       ← Global styles
│   │   ├── index.css     ← Base styles
│   │   ├── services/
│   │   │   └── api.js    ← API calls
│   │   ├── context/
│   │   │   └── SocketContext.jsx ← Socket.io
```

```
│   │   ├──── components/
│   │   │   ├──── DesignDetailModal.jsx
│   │   │   ├──── ImageCropper.jsx + .css
│   │   │   ├──── Modal.css
│   │   │   ├──── ToastProvider.jsx
│   │   │   └──── UserDetailModal.jsx
│   │   └──── pages/
│   │       ├──── Admin.jsx + .css
│   │       ├──── Dashboard.jsx + .css
│   │       ├──── Designer.jsx
│   │       ├──── DesignerProfile.jsx + .css
│   │       ├──── Home.jsx
│   │       ├──── Login.jsx
│   │       ├──── Path.jsx
│   │       ├──── Preview.jsx
│   │       ├──── Profile.jsx + .css
│   │       └──── Register.jsx
│   ├──── public/          ← Static images (should optimize)
│   ├──── .env             ← Frontend env vars
│   └──── vite.config.js
├──── DATABASE/
│   ├──── backup.sql
│   └──── schema.sql
```

└──── package.json ← Root package.json

```

---

I'll help you write a comprehensive technical report about your Aphronique website. Let me break this down systematically.

---

# Aphronique Fashion Platform — Technical Report

## 1. Database Architecture

### What is a Database?

A database is an organized collection of data that your website uses to store, retrieve, and manage information. Aphronique uses **MySQL** (specifically **TiDB Cloud**, a distributed MySQL-compatible database) to store all user data, designs, ratings, and activity logs.

### Database Name

Your database is called **`runway_db`** (you kept the original name even though the project is now called Aphronique).

---

## 2. Tables Overview

Your database has **10 tables**. Here's what each one does:

| Table | Purpose | Records |
```

|-------|---------|---------|

| `users` | Stores all registered accounts (designers, visitors, admins) | 14 users |

| `designs` | Stores fashion design posts with images | 12 designs |

| `likes` | Tracks which users liked which designs | 51 likes |

| `ratings` | Stores star ratings (1-5) given to designs | 6 ratings |

| `follows` | Tracks designer followers | 8 follows |

| `activity_logs` | Records every action (login, upload, like, etc.) | 366 activities |

| `moderation_logs` | For admin moderation actions | Empty |

| `user_notifications` | Stores user alerts (suspensions, etc.) | Empty |

| `products` | For future e-commerce feature | Empty |

---

3. Primary Keys (PK) — The Unique ID System

What is a Primary Key?

A **Primary Key** is a unique identifier for each record in a table. It ensures no two rows have the same ID. In your database, every table uses **`AUTO_INCREMENT`** integers as primary keys.

Examples from Your Database:

`users` table:

```
user_id | username

--------|----------

1       | littlemingming

2       | admin

3       | Jhamirex Reguero T.

8       | admin2
```

9 | awdawdawd

10 | designer123

```

- `user_id = 1` is "littlemingming"

- `user_id = 2` is "admin"

- `user_id = 3` is "Jhamirex Reguero T."

**`designs` table:**

```

design_id | title

----------|----------

1 | IRySjjyjyjy

2 | image

7 | adadassss

26 | d

```

**Why the gaps?** (1, 2, 7, 26...)

When a design is deleted, its ID is **not reused**. The auto-increment counter keeps going up. Design #3, #4, #5, #6 were deleted, so the next new design got ID #7.

---

## 4. Foreign Keys (FK) — Linking Tables Together

### What is a Foreign Key?

A **Foreign Key** creates a relationship between two tables. It ensures data integrity — you can't reference a user or design that doesn't exist.
```

Your Foreign Key Relationships:

Child Table	Foreign Key	References	Meaning
`designs.designer_id`	→	`users.user_id`	Every design belongs to a user
`likes.user_id`	→	`users.user_id`	Every like is from a user
`likes.design_id`	→	`designs.design_id`	Every like is for a design
`ratings.user_id`	→	`users.user_id`	Every rating is from a user
`ratings.design_id`	→	`designs.design_id`	Every rating is for a design
`follows.follower_id`	→	`users.user_id`	Follower is a user
`follows.designer_id`	→	`users.user_id`	Designer being followed is a user
`activity_logs.user_id`	→	`users.user_id`	Every log entry belongs to a user

Example in Action:

```
Design #26 ("d") has designer_id = 13
→ This means user #13 ("Jhamirex Reguero") created it
```

If you try to insert a design with `designer_id = 999`, MySQL will **reject it** because user #999 doesn't exist. This prevents "orphan" data.

---

5. How Registration Works

Step-by-Step Process:

1. User fills the form:

- First Name: "Jhamir"
- Last Name: "Reguero"
- Email: "jhamir@yahoo.com"
- Password: "********"
- Role: "designer" or "visitor"
- Gender: "male" or "female"
- Date of Birth: "2009-10-17"

2. Frontend sends data to backend:

```javascript
POST https://aphronique-api.onrender.com/api/register
{
  username: "Jhamir Reguero",      // Generated from first + last name
  first_name: "Jhamir",
  last_name: "Reguero",
  email: "jhamir@yahoo.com",
  password: "********",
  role: "designer",
  gender: "male",
  date_of_birth: "2009-10-17"
}
```

3. Backend processes:

- Checks if email already exists (`SELECT * FROM users WHERE email = ?`)

- If yes → Returns 400 "Email already exists"

- If no → Hashes the password using **bcrypt**

- Generates username from first + last name

- Validates role (must be 'visitor', 'designer', or 'admin')

- Inserts into `users` table

- Returns success with new `user_id`

4. Password Hashing:

Your passwords are NOT stored as plain text. They are hashed:

```
Original: "mypassword123"

Stored: "$2b$10$HjepzvEPAAdWejmhZmgZzuoyAk5gBDiE6DKtVPpOpZsIHqjwfv2LC"
```

This is a one-way encryption. Even if hackers access the database, they cannot reverse the hash.

---

6. How Login Works

Step-by-Step:

1. User enters credentials:

- Email: "admin@aphronique.com"

- Password: "********"

2. Frontend sends:

```javascript
POST https://aphronique-api.onrender.com/api/login
```

{

email: "admin@aphronique.com",

password: "********"

}

```

**3. Backend processes:**

```javascript

// 1. Find user by email

SELECT * FROM users WHERE email = ?

// 2. Check if account is suspended

if (user.status === 'suspended') → Block login

// 3. Compare password using bcrypt

const match = await bcrypt.compare(inputPassword, hashedPassword)

// 4. If match → Generate JWT token

const token = generateToken(user)

// 5. Log the activity

INSERT INTO activity_logs (user_id, action_type) VALUES (?, 'login')

// 6. Return user data + token

```

4. Token System (JWT):

After login, the backend sends a **JWT token**:
```

```json
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user_id": 2,
  "username": "admin",
  "role": "admin"
}
```

This token is stored in the browser's `localStorage`. Every subsequent request includes this token to prove the user is logged in.

---

7. Why Admin Accounts Are NOT Registered Through the Website

Looking at your database:

user_id	username	email	How Created
1	littlemingming	littlemingming@gmail.com	Registered via website
2	**admin**	**admin@aphronique.com**	**Inserted directly into database**
8	**admin2**	**admin2@runway.com**	**Inserted directly into database**

The Difference:

Regular Users:
- Created through the registration form

- Backend runs `/api/register` route

- Role is limited to 'visitor' or 'designer'

- Password is hashed by bcrypt

Admin Accounts:

- Created by running SQL `INSERT` statements directly in MySQL Workbench

- Or imported from the original database dump

- Role is set to 'admin' manually

- Cannot be created through the website's registration form

Why?

Your registration code validates the role:

```javascript
const validRoles = ['visitor', 'designer', 'admin'];

const userRole = validRoles.includes(role) ? role : 'visitor';
```

Even though 'admin' is technically in the valid list, your frontend registration form only offers two buttons:

- **DESIGNER** → sends `role: "designer"`

- **VISITOR** → sends `role: "visitor"`

There is no "ADMIN" option on the registration page. This is a **security design** — you don't want random people creating admin accounts.

How to Create an Admin:

```sql
INSERT INTO users (username, email, password, role)

VALUES ('newadmin', 'admin@aphronique.com', '$2b$10$...hashedpassword...', 'admin');
```

```

---

## 8. The Unique Numbers Explained

### Auto-Increment IDs:

Every table has an `AUTO_INCREMENT` primary key. This means:

- MySQL automatically assigns the next number
- Numbers never decrease (even if records are deleted)
- Ensures every record has a unique identifier

### Example from `activity_logs`:

```
log_id	user_id	action_type
1	3	login
2	3	update_profile
...
366 | 2 | login
```

- `log_id` goes from 1 to 366 (and counting)
- `user_id` references who did the action
- You can see user #3 (Jhamirex) has logged in many times

### Composite Unique Constraints:

Some tables prevent duplicate relationships:
```

`likes` table:

```sql
UNIQUE KEY `unique_like` (`user_id`, `design_id`)
```

This means a user can only like a design **once**. If user #1 tries to like design #2 again, MySQL rejects it.

`follows` table:

```sql
UNIQUE KEY `unique_follow` (`follower_id`, `designer_id`)
```

You can't follow the same designer twice.

---

9. How the Website Functions Work

Discovery Feed (`/api/designs`):

```sql
SELECT d.*, u.username as designer_name,
    (SELECT AVG(rating) FROM ratings WHERE design_id = d.design_id) as avg_rating,
    (SELECT COUNT(*) FROM likes WHERE design_id = d.design_id) as like_count
FROM designs d
JOIN users u ON d.designer_id = u.user_id
ORDER BY d.created_at DESC
```

This query:

1. Gets all designs

2. Joins with users to get designer names

3. Calculates average rating (subquery)

4. Counts likes (subquery)

5. Sorts by newest first

Profile Page (`/api/users/:id`):

```sql
SELECT * FROM users WHERE user_id = ?
```

Plus:

```sql
SELECT COUNT(*) FROM follows WHERE designer_id = ?
```

To show follower count.

Upload Design (`POST /api/designs`):

1. User selects image file

2. Frontend sends to backend with `multipart/form-data`

3. Backend saves image:

  - **Production (Render)**: Uploads to **Cloudinary** (cloud storage)

  - **Local development**: Saves to `backend/uploads/` folder

4. Inserts design record into database

5. Emits real-time notification via Socket.IO

---

10. Image Storage: Two Systems

| Environment | Storage | URL Example |

|-------------|---------|-------------|

| Local dev | `backend/uploads/` folder | `http://localhost:5000/uploads/123.jpg` |

| Production (Render) | **Cloudinary** cloud | `https://res.cloudinary.com/.../aphronique/123.jpg` |

Why Old Images Are Broken:

Your original designs (IDs 1-26) were uploaded locally. Their `image_url` in the database is:

```
/uploads/1776179553400-730695335.jpg
```

On Render, this becomes:

```
https://aphronique-api.onrender.com/uploads/1776179553400-730695335.jpg
```

But the file only exists on your **local computer**, not on Render's server. That's why they show as broken images.

New uploads work because they go to Cloudinary, which stores images permanently in the cloud.

---

11. Security Features

| Feature | Implementation |

|---------|---------------|

| Password hashing | bcrypt (salt + hash) |

| CORS protection | Only allows specific domains |

| JWT tokens | Signed tokens for authentication |

| SQL injection prevention | Parameterized queries (`?` placeholders) |

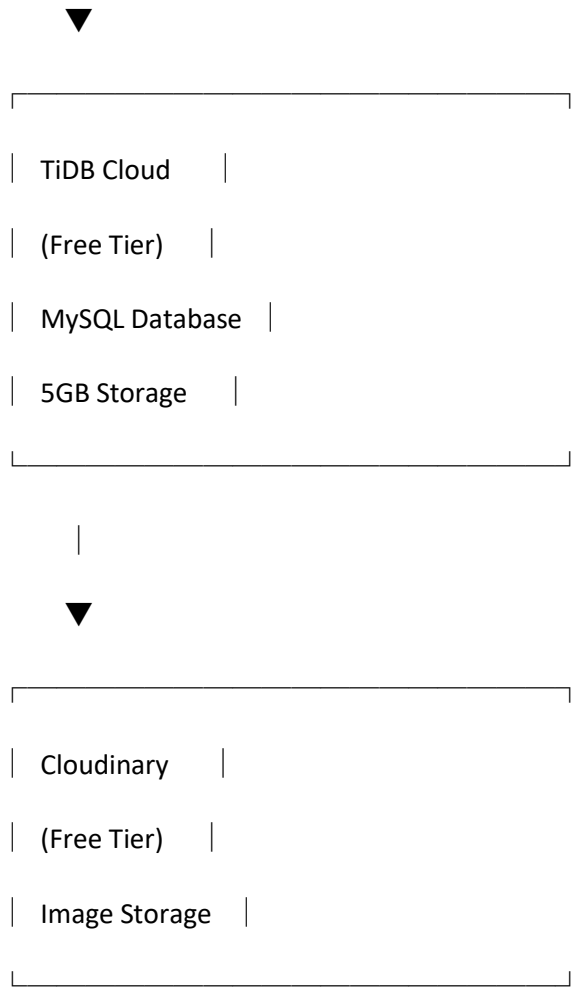
| File upload validation | Only images allowed, max 5MB |

| Role-based access | Admin/Designer/Visitor permissions |

---

12. Deployment Architecture

```
┌──────────────────────────────────┐        ┌──────────────────────────────────┐
│   User Browser   │ ◄──────────► │   Vercel (Free)   │
│                  │ ◄──────────  │   Frontend: React │
└──────────────────────────────────┘        │   URL: aphronique │
                                            │   .vercel.app │
                                            └──────────────────────────────────┘
                                                          │
                                                          ▼
                                            ┌──────────────────────────────────┐
                                            │   Render (Free)   │
                                            │   Backend: Node   │
                                            │   URL: aphronique │
                                            │   -api.onrender   │
                                            │   .com            │
                                            └──────────────────────────────────┘
                                                          │
```




```

---

## Summary for Your Report

**Aphronique** is a full-stack fashion social platform built with:
- **Frontend**: React + Vite (deployed on Vercel)
- **Backend**: Node.js + Express (deployed on Render)
- **Database**: TiDB Cloud (MySQL-compatible, distributed)
- **Image Storage**: Cloudinary (cloud-based CDN)
```

- **Authentication**: JWT tokens with bcrypt password hashing

The database uses relational design with primary keys, foreign keys, and unique constraints to maintain data integrity. Admin accounts are pre-seeded directly into the database for security, while regular users register through the website with roles limited to 'visitor' or 'designer'.

Is there any specific section you want me to expand on for your report?